

Reflection: used to inspect and manipulate objects, and types at runtime

Expression Trees represents code in a tree structure and enable dynamic code manipulation

Analyze Redundancy:

3 separate methods but have same patterns: get 2 parameters, log to console

Rewrite:

```
internal class Logger
{
    3 references
    public void Log(string message, params object[] args)
    {
        Console.WriteLine(FormatLogMessage(message, args));
    }

    1 reference
    private string FormatLogMessage(string message, object[] args)
    {
        for (int i = 0; i < args.Length; i++)
        {
            message = message.Replace($"{{{i}}}", args[i]?.ToString());
        }
        return message;
    }
}
```

Identify DRY violations:

2 Repositories have same get all format

```
public class GenericRepository<T> where T : new()
{
    private readonly SqlConnection _connection;

    0 references
    public GenericRepository(SqlConnection connection)
    {
        _connection = connection;
    }

    0 references
    public List<T> GetAll(string tableName, Func<SqlDataReader, T> mapFunction)
    {
        var resultList = new List<T>();
        var command = new SqlCommand($"SELECT * FROM {tableName}", _connection);
        var reader = command.ExecuteReader();

        while (reader.Read())
        {
            resultList.Add(mapFunction(reader));
        }
        return resultList;
    }
}
```

Identify Redundancy:

Rewrite:

```
using Microsoft.AspNetCore.Mvc;
using System;
using System.Reflection;
using System.Runtime.InteropServices;

[ApiController]
[Route("api/[controller]")]
0 references
public class GenericController<T> : ControllerBase where T : class, new()
{
    [HttpPost]
    0 references
    public IActionResult CreateEntity([FromBody] T entity)
    {
        if (entity == null)
            return BadRequest("Invalid data");

        // Use Reflection to get the "Name" property dynamically
        PropertyInfo nameProperty = typeof(T).GetProperty("Name");

        if (nameProperty == null)
            return BadRequest("Entity must have a Name property.");

        string name = nameProperty.GetValue(entity) as string;

        if (string.IsNullOrEmpty(name))
            return BadRequest("Invalid data: Name is required.");

        return Ok($"{typeof(T).Name} {name} created successfully.");
    }
}
```

The **Strategy Pattern** defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime without modifying the client code.

The **Factory Pattern** is used to create objects **without specifying their concrete class**. Instead, a factory method is responsible for object creation based on input parameters.

Identify DRY violations: 3 separate methods but have same patterns: get 1 parameter, log to console

Rewrite:

```
namespace Lab6.Interface
{
    5 references
    internal interface IPayment
    {
        4 references
        void ProcessPayment(double amount);
    }
}
```

```

namespace Lab6.Model
{
    1 reference
    public class CreditCardPayment : IPayment
    {
        2 references
        public void ProcessPayment(double amount)
        {
            Console.WriteLine($"Processing credit card payment: {amount}");
        }
    }

    1 reference
    public class PayPalPayment : IPayment
    {
        2 references
        public void ProcessPayment(double amount)
        {
            Console.WriteLine($"Processing PayPal payment: {amount}");
        }
    }

    1 reference
    public class CryptoPayment : IPayment
    {
        2 references
        public void ProcessPayment(double amount)
        {
            Console.WriteLine($"Processing cryptocurrency payment: {amount}");
        }
    }
}

```

```

namespace Lab6.Service
{
    0 references
    public class PaymentService
    {
        0 references
        public void ProcessPayment(string method, double amount)
        {
            IPayment paymentStrategy = FactoryPayment.GetPaymentMethod(method);
            paymentStrategy.ProcessPayment(amount);
        }
    }
}

```

```
namespace Lab6.Model
{
    1 reference
    internal class FactoryPayment
    {
        1 reference
        public static IPayment GetPaymentMethod(string method)
        {
            return method.ToLower() switch
            {
                "creditcard" => new CreditCardPayment(),
                "paypal" => new PayPalPayment(),
                "crypto" => new CryptoPayment(),
                _ => throw new ArgumentException("Invalid payment method"),
            };
        }
    }
}
```

Identify Redundancy: 2 services have get 1 item pattern

```
namespace Lab6.Service
{
    0 references
    internal class BaseService
    {
        private Dictionary<int, string> _cache = new Dictionary<int, string>();

        0 references
        public string GetItem(int id)
        {
            if (_cache.ContainsKey(id))
            {
                Console.WriteLine("Fetching from cache...");
                return _cache[id];
            }

            string item = $"Item {id}";
            _cache[id] = item;
            Console.WriteLine("Fetching from cache...");
            return item;
        }
    }
}
```

A scalable and maintainable **Library Management System** in .NET should follow a multi-project architecture with six key projects: **Library.Core** (domain models and business rules), **Library.Infrastructure** (data access with EF Core), **Library.Application** (service layer with DTOs and business logic), **Library.API** (ASP.NET Core Web API for client interaction), **Library.Web** (frontend using Blazor, Angular, or React), and **Library.Shared** (common utilities like logging and helpers). This modular structure ensures **separation of concerns**, easy upgrades, and high reusability while enabling independent scaling of different components.